

BharatStandards.AI

Master Technical & Teacher Guide

For SIH Judges, Mentors, and Engineers

How to Use This PDF

This guide is designed to take a reader from **absolute zero knowledge** to **complete technical mastery** of the BharatStandards.AI prototype.

Recommended Learning Order: * **Day/Session 1:** Problem + BIS + procurement (Levels 0-1) * **Day/Session 2:** Architecture + frontend/backend (Levels 2-4) * **Day/Session 3:** Extraction + embeddings + FAISS (Levels 5-7) * **Day/Session 4:** Matching + reranking + exact citation (Levels 8-9) * **Day/Session 5:** QCO + certification + provenance (Levels 10-12, 15) * **Day/Session 6:** Gap analysis + fixer (Levels 13-14, 16) * **Day/Session 7:** Demo + judge Q&A + Pitches (Examples, Cheat Sheet)

LEVEL 0 — ZERO KNOWLEDGE

What is the problem?

Imagine the government wants to buy **cement** to build a bridge. The procurement officer writes a tender (a shopping list for the government) that says: *"Supply Ordinary Portland Cement, 43 Grade..."*

Before they can publish this tender, they must ensure the cement matches India's official safety and quality rules. They need to know: 1. Which specific **Indian Standard (IS)** applies to this cement? 2. Is that standard still active, or is it outdated? 3. Does the government

mandate a quality mark (like the ISI mark) for this cement? (This is called a **QCO** - Quality Control Order). 4. Are there specific tests the cement must pass? 5. Did I forget to mention important technical details in my tender?

That is the problem BharatStandards.AI is trying to assist with. It is an AI-assisted tool that reads the tender, finds the right standard from thousands of options, checks the rules, and helps the officer fix mistakes.

LEVEL 1 — UNDERSTAND THE PROBLEM STATEMENT (PS)

1. The Problem Statement

The Smart India Hackathon (SIH) problem asks for an **"AI-Powered Recommendation Engine for Applicable Indian Standards."**

2. Why the problem exists

There are over 20,000 Bureau of Indian Standards (BIS) documents. Finding the right one using a simple keyword search on Google or the BIS website is incredibly hard. If you search "steel", you might get 500 results ranging from steel pipes to steel surgical knives.

3. Who faces the problem

Government buyers, procurement officers, and engineers drafting technical specifications.

4. What happens without the system

Officers spend hours manually searching PDFs. Sometimes they guess incorrectly, use an outdated standard, or forget a mandatory QCO, leading to illegal procurements or dangerous construction.

5. What our system changes

It automates the discovery. The officer types what they want, and the AI reads the intent, finds the exact standard, and surfaces the compliance evidence instantly.

6. Expected Input

Free text or a PDF describing the technical requirement (e.g., *"Supply steel pipes for water supply"*).

7. Expected Output

A structured recommendation showing the exact Indian Standard, its Quality Control Order (QCO) status, and any gaps in the user's input.

LEVEL 2 — UNDERSTAND THE WHOLE SYSTEM

Here is the complete end-to-end flowchart of the **actual implemented prototype**:

```
[ USER INPUT ]
  ↓ (Tender PDF / Free Text)
[ TEXT EXTRACTION ] (PyPDF2 for PDFs)
  ↓
[ REQUIREMENT EXTRACTION ] (Rule-based Regex/Heuristics)
  ↓ (Identifies Product, Form, Material, Parameters)
[ EMBEDDING GENERATION ] (Sentence-Transformers all-mpnet-base-v2)
  ↓ (Text to numerical vectors)
[ VECTOR SEARCH ] (FAISS Index)
  ↓ (Finds top candidate standards)
[ SEMANTIC MATCHING ] (Cosine Similarity + Keyword Overlap)
  ↓ (Base Score)
[ RELEVANCE FILTERING & SCOPE RERANKING ]
  ↓ (Drops mismatching product forms, ensures >0.45 score)
[ EXACT CITATION HANDLING ] (If user explicitly typed an IS number)
  ↓
[ COMPLIANCE ENRICHMENT ] (Local JSON Database)
  ↓ (Adds QCO, Certification, Version metadata)
```

```
[ NORMATIVE REFERENCES ] (Local JSON Graph)
    ↓
[ GAP ANALYSIS ] (Checks if user missed required parameters)
    ↓
[ FIX SUGGESTIONS ] (Generates DRAFT revised text)
    ↓
[ PROVENANCE/EVIDENCE ] (Attaches exact match reasons)
    ↓
[ DASHBOARD UI ] (Vanilla JS displays final JSON response)
```

(Note: Every stage shown above is currently fully implemented in the prototype's codebase).

LEVEL 3 — FRONTEND

What is a Frontend?

The frontend is the visual website you interact with in your web browser (Chrome, Edge). It consists of HTML (structure), CSS (styling), and JavaScript (logic).

How does it work?

1. **Browser:** You open `http://127.0.0.1:8000/`.
2. **Frontend:** The browser loads our Vanilla JavaScript app.
3. **API Request:** When you click "Analyze", the frontend sends an HTTP `POST` request (a digital message) containing your text to the backend.
4. **Backend:** The server processes the AI logic.
5. **Response:** The server sends back a `JSON` file (a structured data format).
6. **UI:** The frontend reads the JSON and paints the beautiful dashboard on your screen.

How does OUR project use it?

Our prototype does NOT use heavy frameworks like React or Angular. It is a lightweight, dependency-free Vanilla JS application (`app.js`). It dynamically communicates with our backend via the `/api/v1/analyze` API endpoint using native JavaScript `fetch()`

promises. There is NO fake or mock data; everything on the screen comes live from the backend AI.

LEVEL 4 — BACKEND

What is a Backend?

The backend is the "brain" running on the server. It handles the database, the AI models, and the business rules.

How does it work?

When the frontend asks for an analysis, the request hits an **API Endpoint** (a specific URL address meant for computer-to-computer communication).

How does OUR project use it?

We use **FastAPI**, a modern, ultra-fast Python web framework.

The Pipeline Flow (/api/v1/analyze): 1. **Request:** Receives `{"text": "Supply Cement..."}`. 2. **Extractor (extractor.py):** Detects it is "Cement". 3. **Matcher (matcher.py) & Vector DB (vector_db.py):** Asks the FAISS database for standards related to Cement. 4. **Reranker (reranker.py):** Filters the candidates and calculates a final System Match Score. 5. **Compliance (compliance.py):** Attaches QCO rules. 6. **Gap Analyzer (gap_analysis.py):** Checks for missing data. 7. **Fixer (fixer.py):** Suggests better text. 8. **Response:** Sends the final JSON back to the frontend.

LEVEL 5 — REQUIREMENT EXTRACTION

What is requirement extraction?

It is the process of taking messy, human-written text and breaking it down into structured categories a computer can understand.

Example

INPUT: "Supply Ordinary Portland Cement, 43 Grade, conforming to IS 8112:2013." **OUTPUT:** * Product = Cement * Type = Ordinary Portland Cement * Grade = 43 * Explicit IS = IS 8112:2013

How does OUR project use it?

Our prototype currently uses a **Rule-based Heuristic Extractor** (`extractor.py`). It does *not* use a Large Language Model (LLM) for this stage. It uses highly optimized Regular Expressions (Regex) and vocabulary mapping to detect forms (like "pipe", "bar") and materials (like "steel", "plastic").

Crucially, if a user inputs multiple products (e.g., *"Supply steel pipes, electrical panels and safety helmets"*), our extractor dynamically clones the context and safely splits it into 3 completely isolated requirements. This strictly prevents "cross-contamination" where an electrical standard accidentally gets applied to a steel pipe.

LEVEL 6 — EMBEDDINGS

What is it?

Computers cannot read English words; they only understand numbers. An **Embedding** converts text into a long list of numbers (a vector).

How does it work?

Imagine a 3D map. We want to place the word "Apple" near "Banana", but far away from "Car". An embedding model looks at the context of words and assigns them coordinates. Documents with similar meanings are placed closer together in mathematical space.

How does OUR project use it?

We use a local AI model called `sentence-transformers/all-mpnet-base-v2` . It takes a standard's title/scope (e.g., "Specification for Steel Pipes") and turns it into a mathematical

vector with **768 dimensions** (768 coordinates). When a user types a query, we turn their query into a 768-dimension vector and look for the closest matches.

LEVEL 7 — VECTOR SEARCH / FAISS

What is it?

Keyword search (like hitting Ctrl+F) only finds exact word matches. If you search "hydration material", a keyword search will completely miss a document titled "Cement". **Vector Search** compares the mathematical embeddings. Because "hydration material" and "cement" are conceptually similar, their vectors are close together, and vector search will find it.

How does OUR project use it?

We use **FAISS** (Facebook AI Similarity Search). It is a highly optimized library that stores our 584 pre-calculated standard embeddings. Instead of comparing the user's query against 584 documents one by one (which is slow), FAISS uses geometry to instantly return the top 10 closest standard vectors.

LEVEL 8 — MATCHING

What is it?

Once FAISS gives us the closest candidates, we need to evaluate how good they actually are.

How does OUR project use it?

We calculate a **System Match Score**. (*CRITICAL NOTE: Never call this an "Official BIS Score" or a "Compliance Probability". It is purely an internal system ranking score*).

If the user typed a specific IS number in their text (e.g., "conforming to IS 8112:2013"), our system bypasses semantic search entirely. It directly looks up that exact standard in the

local database and tags it as an **EXACT CITATION**.

If the system cannot find enough evidence, or the score falls below `0.45`, it yields **NOT VERIFIED**.

LEVEL 9 — RERANKING

Why do we need it?

Simple semantic similarity is dangerous in procurement. If a user asks for "Steel pipes", a generic vector search might return "Steel reinforcement bars" with a very high score (e.g., 85%) because both share the word "steel", belong to the construction domain, and are highly related semantically. But providing bars when pipes are requested is a catastrophic procurement failure.

How does OUR project use it?

We built a **Scope-Aware Reranker** (`reranker.py`). After the initial semantic score is generated, the reranker acts as a strict relevance gate. If the user's extracted requirement explicitly specifies a form (e.g., `pipe`), the reranker scans the candidate standard's metadata. If the candidate standard is strictly about `bars` and explicitly incompatible with `pipes`, the reranker applies a **Scope Conflict Penalty**. It forcibly reduces the System Match Score to `0.40`, causing the standard to fail the `0.45` relevance gate and be rejected.

LEVEL 10 — COMPLIANCE

What is compliance in our prototype?

It is the enrichment of a recommended standard with critical regulatory metadata. We provide: * **Status**: Is the standard Active or Withdrawn? * **Version/Amendments**: What is the latest year of publication? * **Certification**: Does it require an ISI mark?

Crucial Distinction

Our system provides "system-gathered evidence". It does **NOT** make an "official legal determination." The human officer uses our system's evidence to make the final legal decision.

LEVEL 11 — QCO (Quality Control Order)

What is a QCO?

A Quality Control Order is a mandatory regulation issued by the Government of India. If a product is under a QCO, it is **illegal** to manufacture, import, or sell it without the official BIS ISI mark.

Why does it matter in procurement?

If a government buyer publishes a tender for a product without realizing it falls under a QCO, they might accidentally buy substandard, illegal goods.

How does OUR project use it?

We have an offline, local mapping file (`qco_mapping.json`). When our system recommends an Indian Standard, it checks this file. If the standard is mapped to a QCO, the frontend alerts the user with a bright red warning: **"Under Mandatory Certification."** (*Note: Because our data is local, the user is reminded to verify this against the latest official government Gazette notifications*).

LEVEL 12 — NORMATIVE REFERENCES

What is a normative reference?

When you read a main standard (e.g., IS 8112 for Cement), it might say, "To test the strength of this cement, follow the rules in IS 4031." IS 4031 is a **Normative Reference**. It is

indispensable; you cannot comply with the main standard without also complying with its normative references.

How does OUR project use it?

We maintain a local `normative_graph.json` file. When the backend returns a recommendation, it checks this graph and lists any related testing or material standards under the "Related/Normative Standards" section in the UI.

LEVEL 13 — GAP ANALYSIS

What is a gap?

A gap is missing or dangerously vague information in the user's tender. * **Vague:** "Supply high quality cement." (No grade or type specified). * **Specific:** "Supply Ordinary Portland Cement, 43 Grade, conforming to IS 8112:2013."

How does OUR project use it?

The `gap_analysis.py` module compares what the `extractor.py` found against what the system expects. Currently, it detects: 1. **Vague Requirement:** The user typed fewer than 3 words (e.g., "Buy cement"). 2. **Missing Core Parameters:** The recommended standard requires a specific measurement or grade, but the user didn't provide one. 3. **Unverified Exact Citation:** The user cited a standard that does not exist in our local knowledge base.

LEVEL 14 — AI FIXER

What does it do?

When a gap is detected, the `fixer.py` module generates a **Draft Revision** of the user's text.

Example

- **User Input:** "Supply Cement"
- **Gap Detected:** Missing exact IS citation and grade.
- **Fixer Output:** "Consider specifying the exact grade and IS number (e.g., Supply 43 Grade Ordinary Portland Cement conforming to IS 8112:2013)."

Important Disclaimer

The fixer is a **drafting assistant**. It is not a legal authority dictating procurement law.

LEVEL 15 — PROVENANCE

What is it?

Provenance means: *"Where did this information come from?"*

Why does it matter?

Government officers cannot trust a "black box" AI score. If the AI says "Score: 0.93", the officer needs to know *why*.

How does OUR project use it?

Every single recommendation from our API includes a `match_reasons` array. The system explicitly outputs the logic. **Example Evidence:** ["Semantic similarity 0.81 between the requirement and the standard's public metadata.", 'Shared keyword/title terms: cement, grade, ordinary.']

LEVEL 16 — NOT VERIFIED

Crucial Distinction

If our system outputs **NOT_VERIFIED**, it does **NOT** mean "The standard does not exist in India." It means: *"The available evidence in our local offline knowledge base was not sufficient to verify a match past our strict safety thresholds."* This is a critical safety feature. It is much better for the AI to honestly say "Not Verified" than to hallucinate and confidently recommend the wrong standard.

PHASE 4 — REAL EXAMPLES (How to explain it)

EXAMPLE 1 — The Perfect Match (Cement)

- **Input:** "Supply 43 Grade Ordinary Portland Cement conforming to IS 8112:2013."
- **Pipeline Execution:** The extractor finds Product: Cement , Explicit IS: IS 8112:2013 . The matcher bypasses semantic search, executes an exact lookup, and returns IS 8112:2013 with a 0.93 score.
- **Evidence:** MATCH TYPE: EXACT_CITATION .

EXAMPLE 2 — The Scope Conflict (Pipes vs Bars)

- **Input:** "Supply steel pipes."
- **Pipeline Execution:** The extractor detects Product: Steel , Form: pipe . Semantic search finds IS 1786 (High Strength Deformed Steel Bars). The Reranker sees pipe vs bar . It triggers the Scope Mismatch penalty. The score is forced to 0.40 . The recommendation fails the 0.45 gate and is safely dropped. Output: NOT_VERIFIED .

EXAMPLE 3 — Safe Multi-Product Split

- **Input:** "Supply steel pipes, electrical panels and safety helmets."
- **Pipeline Execution:** The extractor intelligently splits this into three isolated requirement contexts: Requirement 1 (Steel , form pipe), Requirement 2 (Panels , form panel), Requirement 3 (Helmets , form helmet). Safety Helmets successfully verifies against IS 15907 with a 0.77 score, completely untainted by the steel or electrical context.

EXAMPLE 4 — Vague Unknown

- **Input:** "Unknown industrial machine."
- **Pipeline Execution:** FAISS retrieves random standards based on loose word vectors, but none surpass the semantic relevance baseline. Output: NOT_VERIFIED .

PHASE 5 — COMPLETE TECHNICAL FLOWCHARTS

(Note: In an interactive markdown viewer, these would render as diagrams. Here they are presented as textual flow structures).

1. Overall System Data Flow

User Input → Backend API → Extractor → FAISS Vector DB → Reranker → Compliance/Gap Analysis → Frontend JSON Render

How to read this: This is the high-level bird's-eye view of how a user's typed text travels through the system and returns as a visual dashboard.

2. Frontend → Backend

Browser (app.js) --[HTTP POST /api/v1/analyze with JSON body]--> FastAPI (main.py) --[HTTP 200 OK JSON response]--> DOM Update (HTML)

How to read this: Shows the strict decoupling of our lightweight vanilla JS frontend from our heavy Python backend.

3. Requirement Extraction

Raw Clause → Regex Filters → Extract (Products, Forms, Materials, Citations) → Yield List of Isolated ExtractedRequirements

How to read this: Shows how our deterministic heuristic handles complex multi-product paragraphs safely.

4. Vector Search & Matching

Query Vector (768-dim) → FAISS Index → Top K Candidates → Cosine Similarity + Keyword Overlap → Base Score

How to read this: The mathematical heart of the system where language is compared as numbers.

5. Scope-Aware Reranking

Base Score → Check Required Form → Scan Candidate Title/Metadata for Form
 → If Mismatch → Apply Penalty (Score = 0.40) → Else Keep Score **How to read this:** The critical safety valve preventing "Steel Pipes" from matching "Steel Bars".

PHASE 6 — TECHNICAL WORD GLOSSARY

- **API (Application Programming Interface):**
 - *Simple:* A digital waiter that takes your order to the kitchen and brings food back.
 - *Our project:* FastAPI serving `/api/v1/analyze` .
- **Embedding / Vector:**
 - *Simple:* Converting a word into a GPS coordinate so computers know how "close" two words are.
 - *Our project:* 768-dimensional numbers generated by `sentence-transformers` .
- **FAISS (Vector Database):**
 - *Simple:* A super-fast filing cabinet designed specifically for GPS coordinates (embeddings).
 - *Our project:* Stores our 584 standard embeddings for millisecond retrieval.
- **Provenance:**
 - *Simple:* The digital receipt showing exactly why a decision was made.
 - *Our project:* The `match_reasons` array output by our matcher.
- **QCO (Quality Control Order):**
 - *Simple:* A government rule making an ISI mark legally mandatory.
 - *Our project:* Mapped via `qco_mapping.json` .
- **Heuristic Extraction:**
 - *Simple:* Extracting information using strict, pre-programmed rules (Regex) rather than an unpredictable AI brain (LLM).
 - *Our project:* Used in `extractor.py` to guarantee fast, deterministic parsing without hallucination.

PHASE 7 — PROJECT FILE MAP

- `Backend/app.main.py` : The entry point that boots up the FastAPI server.
- `Backend/app/api/endpoints.py` : Defines the actual URLs (like `/analyze`) the frontend talks to.
- `Backend/app/services/extractor.py` : Houses the regex rules that find products and forms in the text.
- `Backend/app/services/vector_db.py` : Connects to FAISS to find similar standards.
- `Backend/app/services/reranker.py` : Enforces the Scope Conflict Penalty to prevent form mismatches.
- `Backend/data/bis_metadata.json` : The offline database of 584 standards.
- `Frontend/app.js` : The Javascript file that takes user input, calls the backend, and paints the screen.

PHASE 8 — DATA FLOW (The Journey of a Tender)

1. RAW INPUT User types: `"Supply Ordinary Portland Cement, 43 Grade, conforming to IS 8112:2013."`

2. EXTRACTED STRUCTURE The backend parses this into Python objects: `{"product": "Cement", "form": "cement", "cited_standards": ["IS 8112:2013"]}`

3. VECTOR SEARCH / EXACT MATCH System detects the exact citation. It skips the FAISS embedding math and immediately does a database lookup for `IS 8112:2013` .

4. COMPLIANCE ENRICHMENT System checks `qco_mapping.json` . It finds `IS 8112` is under mandatory certification.

5. GAP ANALYSIS System compares input parameters against expected. (No major gaps found).

6. FINAL JSON RESPONSE

```
{
  "requirements": [{"requirement_id": "REQ-001", "product": "Cement"}],
  "recommendations": {
    "REQ-001": [
      {
```

```
    "is_number": "IS 8112:2013",
    "title": "Ordinary Portland Cement, 43 Grade",
    "system_match_score": 0.93,
    "match_reasons": ["MATCH TYPE: EXACT_CITATION..."],
    "compliance": {"qco_status": "Under Mandatory Certification"}
  }
]
}
}
```

7. FRONTEND RENDERING `app.js` reads this JSON and turns the QCO status into a bold red warning badge on the dashboard.

PHASE 9 — API EXPLANATION

Here are the actual endpoints implemented in the FastAPI backend:

METHOD	ENDPOINT	PURPOSE
POST	<code>/api/v1/analyze</code>	The massive end-to-end pipeline. Takes text/pdf, runs extraction, retrieval, reranking, and gaps, returning the final dashboard JSON.
POST	<code>/api/v1/extract</code>	Utility endpoint. Only runs the <code>extractor.py</code> and returns the identified products.
POST	<code>/api/v1/search</code>	Utility endpoint. Takes a query and only runs FAISS vector retrieval.
GET	<code>/api/v1/standards/{is_number}</code>	Database lookup for a single standard's metadata.

PHASE 10 — JUDGE DEMO SCRIPT (3 Minutes)

[0:00] SETUP Open `http://127.0.0.1:8000/` . *What to say:* "Welcome to BharatStandards.AI. This prototype solves the complex problem of translating messy government procurement tenders into structured, verifiable Indian Standards."

[0:30] EXACT MATCH DEMO Paste: `"Supply 43 Grade Ordinary Portland Cement conforming to IS 8112:2013."` Click Analyze. *What to say:* "Here, the officer explicitly cited a standard. Our backend instantaneously extracts the citation, verifies it in our local database, and crucially, surfaces that this cement is under a Mandatory Quality Control Order (QCO), preventing an illegal procurement."

[1:15] SCOPE MISMATCH DEMO Paste: `"Supply steel pipes."` Click Analyze. *What to say:* "This is where traditional AI fails. A normal semantic search sees 'steel' and recommends 'Steel Bars'. Our architecture features a Scope-Aware Reranker. It extracts the form 'pipe', scans the candidate 'bars', identifies the strict mismatch, and penalizes the score, safely returning 'Not Verified' rather than hallucinating the wrong form."

[2:00] MULTI-PRODUCT DEMO Paste: `"Supply steel pipes, electrical panels and safety helmets."` *What to say:* "Tenders often combine items. If we embed this entire sentence, the vectors blend and return garbage. Our heuristic extractor safely splits this into three isolated requirements, completely preventing cross-contamination. Notice how it perfectly verified the Safety Helmet standard without getting confused by the steel."

[2:45] CLOSING *What to say:* "We achieved this without expensive cloud LLMs. We used local Sentence-Transformer embeddings, FAISS vector search, and strict deterministic reranking to ensure speed, absolute privacy, and provenance."

PHASE 11 — JUDGE Q&A

Q1: Is this just a ChatGPT wrapper? *Short Answer:* No. *Deep Answer:* We do not use LLMs for extraction or matching. We built a custom pipeline using deterministic Regex heuristics for extraction, local `sentence-transformers` for embeddings, and FAISS for vector retrieval. It runs 100% offline.

Q2: How do you prevent hallucinations? *Short Answer:* Strict Relevance Gates and Scope Mismatch Penalties. *Deep Answer:* If the semantic FAISS score falls below `0.45` , we drop it.

More importantly, if our Reranker detects the user wants a "pipe" but the standard is for a "bar", it applies a mathematical penalty to forcibly drop the score below the threshold. We output `NOT_VERIFIED` instead of guessing.

Q3: Where does your data come from? Are you connecting to the live BIS server?

Short Answer: Local offline JSON datasets. *Deep Answer:* For the prototype, we compiled a static database of ~584 standards and 201 QCO mappings. We do not currently scrape or ping the live BIS server. In production, this local database would be synced nightly via an official BIS API webhook.

Q4: Can this replace a procurement officer? *Short Answer:* Absolutely not. *Deep Answer:* This is a decision-support system. It provides evidence, match reasons, and QCO alerts. The final legal authority and verification against the official government gazette remain strictly with the human officer.

Q5: Why did you use Sentence-Transformers instead of OpenAI embeddings? *Short Answer:* Privacy, cost, and latency. *Deep Answer:* Government procurement data can be highly sensitive. By running `all-mpnet-base-v2` locally, no tender text ever leaves the government's intranet.

PHASE 12 — EXPLAIN IT IN...

- **30 Seconds (Elevator Pitch):** "We built a local, offline AI search engine for government buyers. You paste in a messy tender document, and it instantly tells you which Indian Standards apply, if they require mandatory ISI certification, and if you forgot any crucial technical details."
 - **Non-Technical:** "It's like an auto-correct and rule-checker for buying things. If the government wants to buy cement, our tool ensures they follow the exact safety laws for that specific type of cement."
 - **Technical:** "A hybrid NLP pipeline utilizing heuristic requirement extraction, FAISS vector retrieval via `all-mpnet-base-v2` embeddings, and a deterministic scope-aware reranking gate to map free-text tenders to a localized JSON graph of BIS standards."
-

PHASE 13 — WHAT WE SHOULD NOT CLAIM (Red Flags)

- **DO NOT SAY:** "100% Accurate"
 - **SAY INSTEAD:** "Highly optimized for our test matrix using strict relevance gates."
 - **DO NOT SAY:** "Official BIS Compliance"
 - **SAY INSTEAD:** "System-provided compliance evidence based on offline mapping."
 - **DO NOT SAY:** "Real-time BIS Database"
 - **SAY INSTEAD:** "Localized prototype knowledge base of ~584 standards."
 - **DO NOT SAY:** "LLM Extraction"
 - **SAY INSTEAD:** "Heuristic / Regex Extraction." (Do not lie about using an LLM if the code uses rule-based extraction).
-

PHASE 14 — CURRENT PROTOTYPE LIMITATIONS

1. **Static Data:** The ~584 standard records and QCO mappings are offline JSON files. They will become outdated without manual updates.
 2. **Regex Fragility:** While fast, the deterministic extraction relies on predefined vocabularies. If a user inputs a completely novel product form not in our regex dictionary, it defaults to a broader semantic search, increasing the risk of false positives.
 3. **PDF Tables:** The PyPDF2 extractor handles raw text but fails on complex tabular data (like dimensional tolerance tables) common in engineering drawings.
-

PHASE 15 — FUTURE PRODUCTION ARCHITECTURE

To move this from a prototype to a National Production System:

1. **Data Synchronization:** Implement a webhook listener to auto-update the JSON/FAISS index whenever BIS publishes an amendment or QCO.
2. **Vision-Language Models (VLM):** Replace PyPDF2 with an enterprise OCR pipeline capable of parsing engineering tables.
3. **Authentication:** Add SAML/OAuth2 for official SSO login.
4. **Audit Logging:** Record every analysis in a secure, immutable database for procurement oversight.

PHASE 16 — FINAL CHEAT SHEET

THE PROJECT IN ONE LINE

An offline, hybrid AI/heuristic recommendation engine that translates free-text procurement specifications into verified Indian Standards and QCO compliance evidence.

THE STACK

- **Backend:** FastAPI, Python, Pytest.
- **AI/Retrieval:** Sentence-Transformers (`all-mpnet-base-v2`), FAISS Vector Database.
- **Frontend:** Vanilla HTML/JS (`fetch` API).
- **Database:** Offline JSON files.

CORE CONCEPTS

- **Extraction:** Deterministic rules separate multi-product sentences to prevent cross-contamination.
- **Embeddings & FAISS:** Converts text to 768-dimension vectors and searches them instantly.
- **Reranking (The Secret Sauce):** Penalizes candidate standards heavily if their form (e.g., "bars") mismatches the user's requested form (e.g., "pipes").
- **Provenance:** Every answer includes explicit `match_reasons` so the user knows exactly why a standard was recommended.
- **NOT VERIFIED:** Means the system safely aborted the recommendation because it failed the threshold or scope check. It does *not* mean the standard doesn't exist.

5 THINGS TO REMEMBER FOR JUDGES

1. **"It runs locally."** (Privacy + Speed).
 2. **"It prevents cross-contamination."** (Splits multi-product sentences).
 3. **"It checks forms."** (Pipe vs Bar penalty).
 4. **"It highlights QCOs."** (Prevents illegal procurements).
 5. **"It is a decision-support tool, not a legal authority."** (Human in the loop).
-

End of Guide.